QUANSER
INNOVATE·EDUCATE

Lab Procedure
Push Button

Introduction

Ensure the following:

1. You have reviewed the Application Guide – Push Button
2. Make sure the **Mechatronic Sensors Trainer** is out of its case; it is powered using the provided USB cable connected to the PC.
    a. The LED next to the USB C port will stay white after the touch screen starts loading.
    b. The load screen with the Quanser Logo should disappear after a few seconds and you should see some evenly spaced crosses on a black background on the touch screen.

Exploring Push Button Inputs and Polarity

1. Launch Visual Studio Code or your preferred Python IDE. If using Visual Studio Code, click on **File** > **Open Folder**..., and browse to the directory containing the python files and lab procedure for the Push Button lab (…/sensors_trainer/1_fundamentals /push_button/hardware/python). Open this directory.
2. Open buttons.py. The device will get initialized in the line:
   ```
   with SensorsTrainer(btn0Pol=1, btn1Pol=1) as sensors:
   ```
3. Open the datasheet for the button located in the push button directory (…/sensors_trainer/1_fundamentals /push_button) and scroll down to the 320.03 section, which contains the part drawing and schematic for the buttons on the sensors trainer. Look at the schematic, which shows that the button is normally open so no current flows between terminals 1 and 3. Pressing the button will complete the connection between the terminals and allow current to flow through.
4. Run the script by clicking the play button  at the top right corner of the code editor. This will open a **`Buttons`** scope displaying the inputs from **`Button 0`** and **`Button 1`**.
5. Observe the **`Buttons`** scope as you press **`Button 0`** and **`Button 1`**. The buttons are located on the right side of the LCD screen in the device. What happens to each signal when each button is pressed or released? What happens when each button is held down?
   NOTE: Do not pay attention to the Y axis numbers, signals were placed like that for simplicity. Instead, focus on whether signals are high or low.

6. Stop the script by clicking back on the terminal in Visual Studio Code, then pressing Ctrl + C, and then clicking enter when prompted in the terminal.

7. In buttons.py, find the initialization line and set **`btn0Pol=0`**. This tells the software to invert the polarity of **`Button 0`** to give us the behavior of a normally closed switch.

8. Save your changes and run the script again. Press the buttons. What happens to each signal when each button is pressed or released? What happens when each button is held down? What did your change in the previous step do? Stop the script.

Implementing Latching Switch

9. The third signal on the scope displays the value of **`buttonLatch`**. Suppose we wanted to use **`Button 1`** as a toggle switch for **`buttonLatch`**. The desired behavior is for the **`buttonLatch`** signal to switch values from low to high when we press **`Button 1`**, and to remain high until the next time that we press **`Button 1`**, switching it back to low. How would you expect the **`buttonLatch`** signal to appear on the scope?

10. Find the section of code under the comment **`# Toggle buttonLatch when Button 1 is pressed`**. Highlight lines 51-62 and use Ctrl + / to uncomment the if statement and the code block under it. Run the script again.

11. Press **`Button 1`** a few times and watch the **`buttonLatch`** signal in the scope plot. What happens to the value of **`buttonLatch`**? Take a screenshot of the scope to show what happens to **`buttonLatch`** when you press **`Button 1`** several times. Stop the script.

12. In the script, after switching the value of **`buttonLatch`**, the value stored in variable **`buttonPressCounter`** is incremented by 1. Add a line of code to print the new values of **`buttonPressCounter`** and **`buttonLatch`** to the terminal each time **`buttonPressCounter`** is incremented. When the sensors trainer reads button inputs, the values are stored in the variable **`buttons`**. You can index them using **`buttons[0]`** and **`buttons[1]`** for buttons 0 and 1 respectively.

13. Run the script. What happens to **`buttonLatch`** when you press **`Button 1`**? What needs to change if we only want to toggle **`buttonLatch`** once each time **`Button 1`** is pressed?

14. Stop the script. Modify the condition in the statement **`if buttons[1] == 1:`** to switch **`buttonLatch`** and increment **`buttonPressCounter`** only once for each time that Button 1 is pressed.
    Hint: Use the variable **`prevButton1`**, which stores the value of **`Button 1`** at the end of each control loop

15. Run the script, until your code shows the expected behaviour. Take a screenshot that show that **`buttonLatch`** successfully toggles only once each time **`Button 1`** is pressed.

16. Stop the script.

Exploring Sampling Time

17. Change the line **`if sampleCounter%4 == 0`**: to **`if sampleCounter%10 == 0`**: which will make the scope update every 10th sample. Run the script.

18. Press **`Button 1`**. Try clicking it multiple times quickly while observing both the printed outputs in the terminal and the scope simultaneously. Is the terminal output consistent with the scope output? How are they different? Why?

19. At the top of the file, look for the line **`frequency = 100 # Hz`**. Change the sampling frequency to 400. And repeat the last step. What do you notice?

20. Stop, save and close your program.

21. Power OFF the Mechatronic Sensors Trainer by disconnecting its USB C cable, disconnect any external sensors and pack them along the base and the USB cables in the provided case.